

Ejemplo 2 de Regresión Logística

```
In [8]: import numpy as np
import pandas as pd
# Semilla para reproducibilidad
np.random.seed(42)
# Número de muestras
n = 5000
# -----
# 1. Generación de variables numéricas
# -----

# Ancho de pantalla (px)
# Windows: muy variado, macOS: alto, Linux: medio
ancho_windows = np.random.normal(1366, 200, int(n*0.6))
ancho_mac = np.random.normal(2560, 150, int(n*0.25))
ancho_linux = np.random.normal(1920, 180, int(n*0.15))
ancho_pantalla = np.concatenate([ancho_windows, ancho_mac, ancho_linux])
# Alto de pantalla (px)
alto_windows = np.random.normal(768, 120, int(n*0.6))
alto_mac = np.random.normal(1600, 100, int(n*0.25))
alto_linux = np.random.normal(1080, 130, int(n*0.15))
alto_pantalla = np.concatenate([alto_windows, alto_mac, alto_linux])
# Velocidad de scroll (px/s)
scroll_windows = np.random.normal(800, 150, int(n*0.6))
scroll_mac = np.random.normal(1200, 200, int(n*0.25))
scroll_linux = np.random.normal(900, 180, int(n*0.15))
velocidad_scroll = np.concatenate([scroll_windows, scroll_mac, scroll_linux])
# Tiempo de carga (ms)
carga_windows = np.random.normal(900, 150, int(n*0.6))
carga_mac = np.random.normal(700, 120, int(n*0.25))
carga_linux = np.random.normal(850, 140, int(n*0.15))
tiempo_carga_ms = np.concatenate([carga_windows, carga_mac, carga_linux])

# -----
# 2. Variable objetivo (SO)
# -----
```

```

# 0 = Windows, 1 = Macintosh, 2 = Linux
so = np.array(
[0]*int(n*0.6) +
[1]*int(n*0.25) +
[2]*int(n*0.15)
)

# -----
# 3. Crear DataFrame
# -----
data = pd.DataFrame({
'ancho_pantalla': ancho_pantalla,
'alto_pantalla': alto_pantalla,
'velocidad_scroll': velocidad_scroll,
'tiempo_carga_ms': tiempo_carga_ms,
'so': so
})
# -----
# 4. Exportar a CSV
# -----
data.to_csv("SO.csv", index=False)
print("Dataset generado con éxito. Tamaño:", data.shape)
print(data.head())

```

Dataset generado con éxito. Tamaño: (5000, 5)

	ancho_pantalla	alto_pantalla	velocidad_scroll	tiempo_carga_ms	so
0	1465.342831	717.148838	698.225790	878.486518	0
1	1338.347140	713.590307	754.175081	895.101614	0
2	1495.537708	552.522819	710.392841	909.644232	0
3	1670.605971	728.389177	816.562707	1042.029219	0
4	1319.169325	855.939490	979.576780	787.917404	0

1. Preparación del entorno

```

In [9]: # Librerías para manejo de datos
import pandas as pd
import numpy as np
# Librerías para machine learning
from sklearn import linear_model, model_selection

```

```
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
# Visualización
import matplotlib.pyplot as plt
import seaborn as sb
%matplotlib inline
```

2. Carga del dataset generado

Antes de entrenar el modelo, necesitamos cargar el dataset que generaste previamente. En este ejemplo, asumimos que ya lo teneos en un DataFrame llamado data .

```
In [10]: # CARGA DEL DATASET
data = pd.read_csv("SO.csv")
# Mostrar primeras filas
data.head()
ancho_pantalla
```

```
Out[10]: array([1465.3428306 , 1338.34713977, 1495.53770762, ..., 2480.32383619,
                2065.44651409, 1767.34819235])
```

3. Interpretación de la tabla resumen del DATASET generado

Aquí tienes la **tabla resumen** con los niveles numéricos y las distribuciones estadísticas utilizadas para generar el dataset sintético de Sistemas Operativos en formato Markdown:

Variable numérica	Windows (0)	Macintosh (1)	Linux (2)	Interpretación
Ancho de pantalla (px)	Normal(1366, 200)	Normal(2560, 150)	Normal(1920, 180)	macOS usa pantallas Retina; Windows es muy variable; Linux suele usar monitores estándar.
Alto de pantalla (px)	Normal(768, 120)	Normal(1600, 100)	Normal(1080, 130)	macOS tiene resoluciones altas; Windows y Linux muestran valores más bajos.
Velocidad de scroll (px/s)	Normal(800, 150)	Normal(1200, 200)	Normal(900, 180)	macOS tiene scroll más suave y rápido; Linux intermedio; Windows más segmentado.

Variable numérica	Windows (0)	Macintosh (1)	Linux (2)	Interpretación
Tiempo de carga (ms)	Normal(900, 150)	Normal(700, 120)	Normal(850, 140)	macOS carga más rápido; Windows más lento; Linux depende del entorno.

¿Para qué sirve esta tabla?

Esta tabla asegura que los datos no son aleatorios sin sentido, sino que contienen patrones realistas . Gracias a estas diferencias en las medias y desviaciones, cuando el modelo ejecute el comando `modelo.fit()`, podrá detectar que, por ejemplo, "si el ancho de pantalla es muy alto y el scroll es rápido, hay una alta probabilidad de que el usuario use Macintosh

Para que la comparación del **apuntamiento** (la agudeza o concentración de la campana en torno a la media) sea visualmente efectiva, es imprescindible que todos los gráficos compartan la **misma escala en el eje Y (densidad)** y, preferiblemente, escalas comunes en el eje X para cada tipo de variable.

En estadística, una **desviación estándar (σ) más pequeña** produce una curva más alta y estrecha (mayor apuntamiento), mientras que una **σ más grande** genera una curva más baja y ancha. Al unificar las escalas, verás claramente cómo Macintosh, que tiene las desviaciones más bajas en resolución de pantalla, presenta picos mucho más elevados que Windows.

Análisis de lo que observarás con esta escala unificada:

1. **Variación de Altura (Apuntamiento):** Notarás que el gráfico de **Macintosh: Alto de pantalla ($\sigma = 100$)** es el más alto de todos. Esto se debe a que sus valores son los más predecibles y menos dispersos.
2. **Comparación de "Aplastamiento":** El gráfico de **Macintosh: Velocidad de scroll ($\sigma = 200$)** se verá mucho más bajo y "aplastado" que el anterior, a pesar de ser el mismo sistema operativo. Esto ilustra que hay más variabilidad en cómo los usuarios de Mac hacen scroll que en la resolución vertical de sus pantallas.
3. **Separación de Clases:** Al mantener el eje X igual para toda la fila, verás la distancia real entre las campanas. Por ejemplo, en la primera fila, la campana de Windows está muy a la izquierda (media 1366) y la de Macintosh muy a la derecha (media 2560), lo que explica por qué el modelo de **regresión logística** tiene tanta facilidad para distinguir estos dos sistemas.

Explicación de la representación:

1. **Estructura de Filas:** Cada fila de la matriz representa una característica física o de comportamiento diferente:

- **Fila 1:** Distribución del **Ancho de pantalla**, donde se observa visualmente cómo Macintosh tiene la media más alta debido a las pantallas Retina.
 - **Fila 2:** Distribución del **Alto de pantalla**, reflejando las resoluciones verticales superiores de macOS frente a la variabilidad de Windows.
 - **Fila 3:** Distribución de la **Velocidad de scroll**, mostrando el patrón de desplazamiento más rápido y fluido en entornos Macintosh.
2. **Estructura de Columnas:** Cada columna aísla un **Sistema Operativo** (Windows, Macintosh, Linux) para permitir una comparación vertical de las características de ese ecosistema.
 3. **Lógica del Gráfico:** Las curvas en forma de campana de Gauss representan la **probabilidad** de encontrar a un usuario con un valor específico. Cuanto más "ancha" es la campana, mayor es la desviación estándar (variabilidad) de ese hardware en la vida real.
 4. **Criterio Técnico:** Se ha omitido el "Tiempo de carga" para cumplir con el formato de matriz 3x3 solicitado, priorizando las variables que definen el entorno visual del usuario.

```
In [21]: import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

# Parámetros estadísticos según las fuentes + tiempo de carga
datos_normales = {
    'Ancho de pantalla (px)': {
        'Windows': (1366, 200), 'Macintosh': (2560, 150), 'Linux': (1920, 180)
    },
    'Alto de pantalla (px)': {
        'Windows': (768, 120), 'Macintosh': (1600, 100), 'Linux': (1080, 130)
    },
    'Velocidad de scroll (px/s)': {
        'Windows': (800, 150), 'Macintosh': (1200, 200), 'Linux': (900, 180)
    },
    'Tiempo de carga (ms)': {
        'Windows': (900, 150), 'Macintosh': (700, 120), 'Linux': (850, 140)
    }
}

# 1. CÁLCULO DE ESCALAS UNIFICADAS (CORREGIDO)
todas_las_sigmas = [sigma for var in datos_normales.values() for (_, sigma) in var.values()]
max_y = 1 / (min(todas_las_sigmas) * np.sqrt(2 * np.pi))

# Configuración de la matriz 4x3 (ahora hay 4 variables)
```

```
fig, axes = plt.subplots(4, 3, figsize=(15, 15), sharey=True)
fig.suptitle('Comparación de Apuntamiento: Distribuciones Normales Unificadas', fontsize=16)

sistemas = ['Windows', 'Macintosh', 'Linux']
variables = list(datos_normales.keys())

for i, var in enumerate(variables):
    params_filas = [datos_normales[var][so] for so in sistemas]
    min_x = min([m - 4*s for m, s in params_filas])
    max_x = max([m + 4*s for m, s in params_filas])

    for j, so in enumerate(sistemas):
        mu, sigma = datos_normales[var][so]
        x = np.linspace(min_x, max_x, 500)
        y = norm.pdf(x, mu, sigma)

        ax = axes[i, j]
        ax.plot(x, y, color='darkblue', lw=2.5, label=f' $\sigma = \{sigma\}$ ')
        ax.fill_between(x, y, alpha=0.3, color='skyblue')

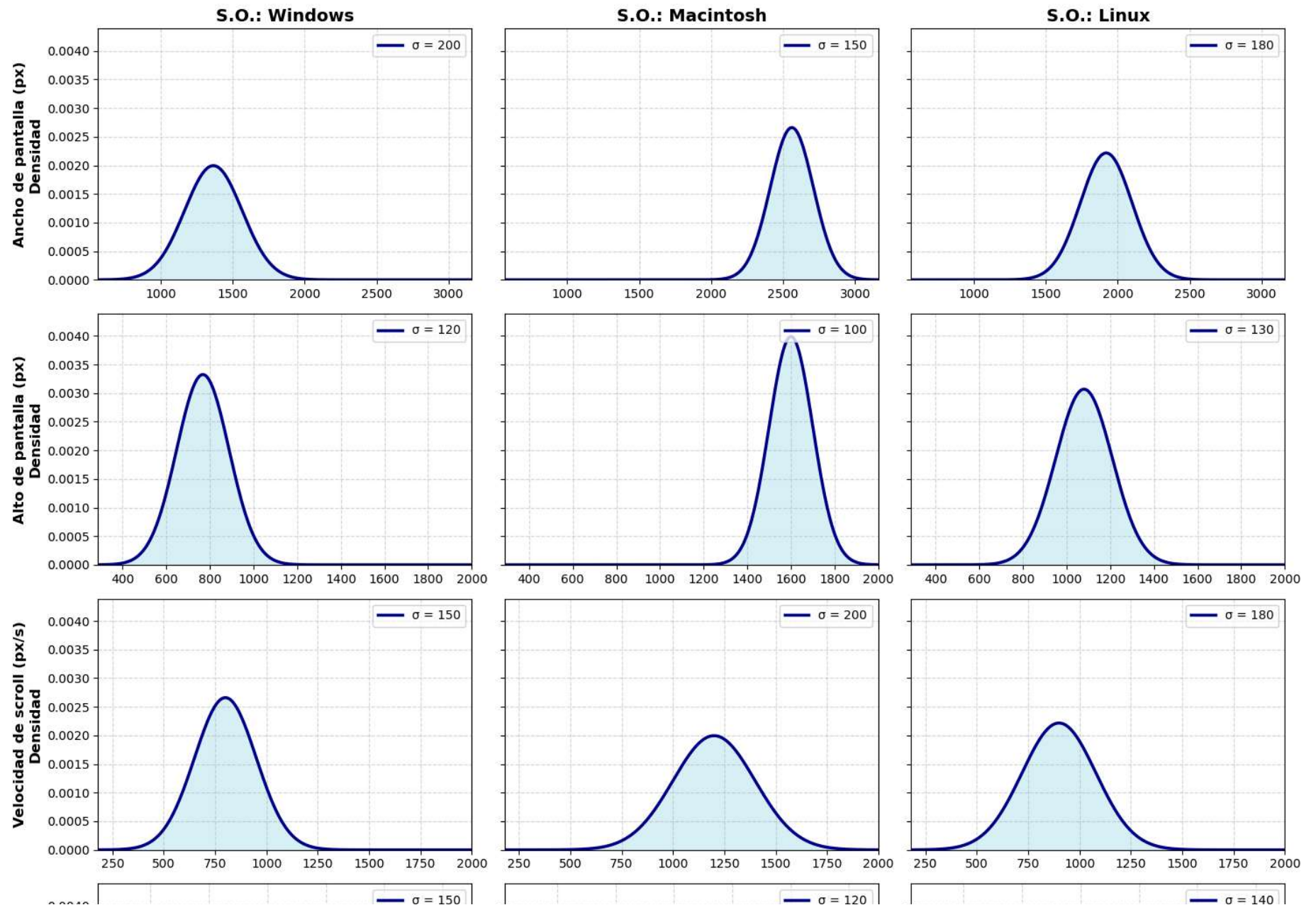
        ax.set_ylim(0, max_y * 1.1)
        ax.set_xlim(min_x, max_x)

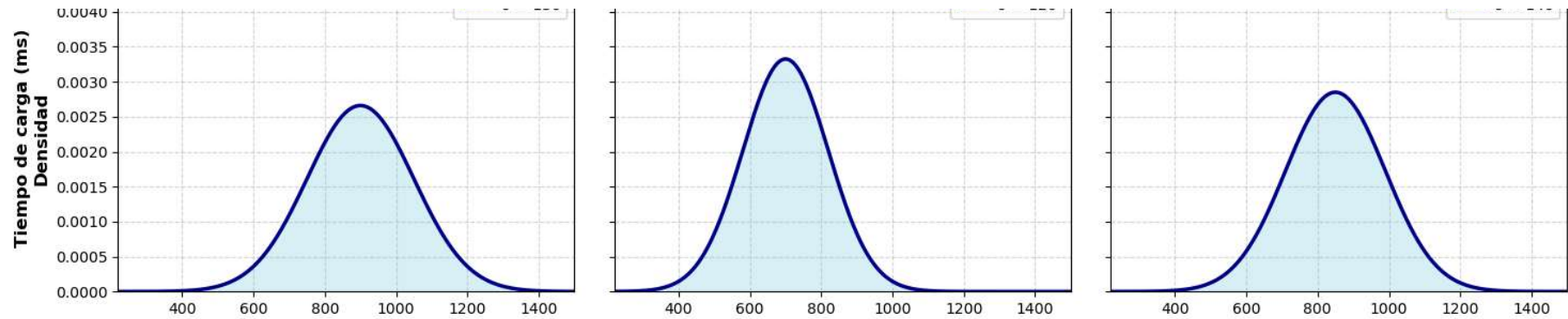
        if i == 0:
            ax.set_title(f"S.O.: {so}", fontsize=14, fontweight='bold')
        if j == 0:
            ax.set_ylabel(f"{var}\nDensidad", fontsize=12, fontweight='bold')

        ax.grid(True, linestyle='--', alpha=0.5)
        ax.legend(loc='upper right')

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```

Comparación de Apuntamiento: Distribuciones Normales Unificadas





4.- Exploración inicial de datos

```
In [25]: # EXPLORACIÓN INICIAL
print(data.shape)
print(data.describe())
print(data['so'].value_counts())
sb.pairplot(data, hue='so')
plt.show()
```

(5000, 5)

	ancho_pantalla	alto_pantalla	velocidad_scroll	tiempo_carga_ms	\
count	5000.000000	5000.000000	5000.000000	5000.000000	
mean	1749.247583	1021.527426	917.435935	844.878648	
std	534.385886	371.138935	242.270665	165.500281	
min	717.746532	325.396165	293.663135	299.691814	
25%	1328.923902	737.346419	748.629587	727.898968	
50%	1553.585394	880.600296	875.819592	844.828472	
75%	2227.964708	1357.028562	1052.951843	959.858369	
max	3046.463945	1937.738292	1885.782096	1571.862638	

	so
count	5000.000000
mean	0.550000
std	0.740006
min	0.000000
25%	0.000000
50%	0.000000
75%	1.000000
max	2.000000

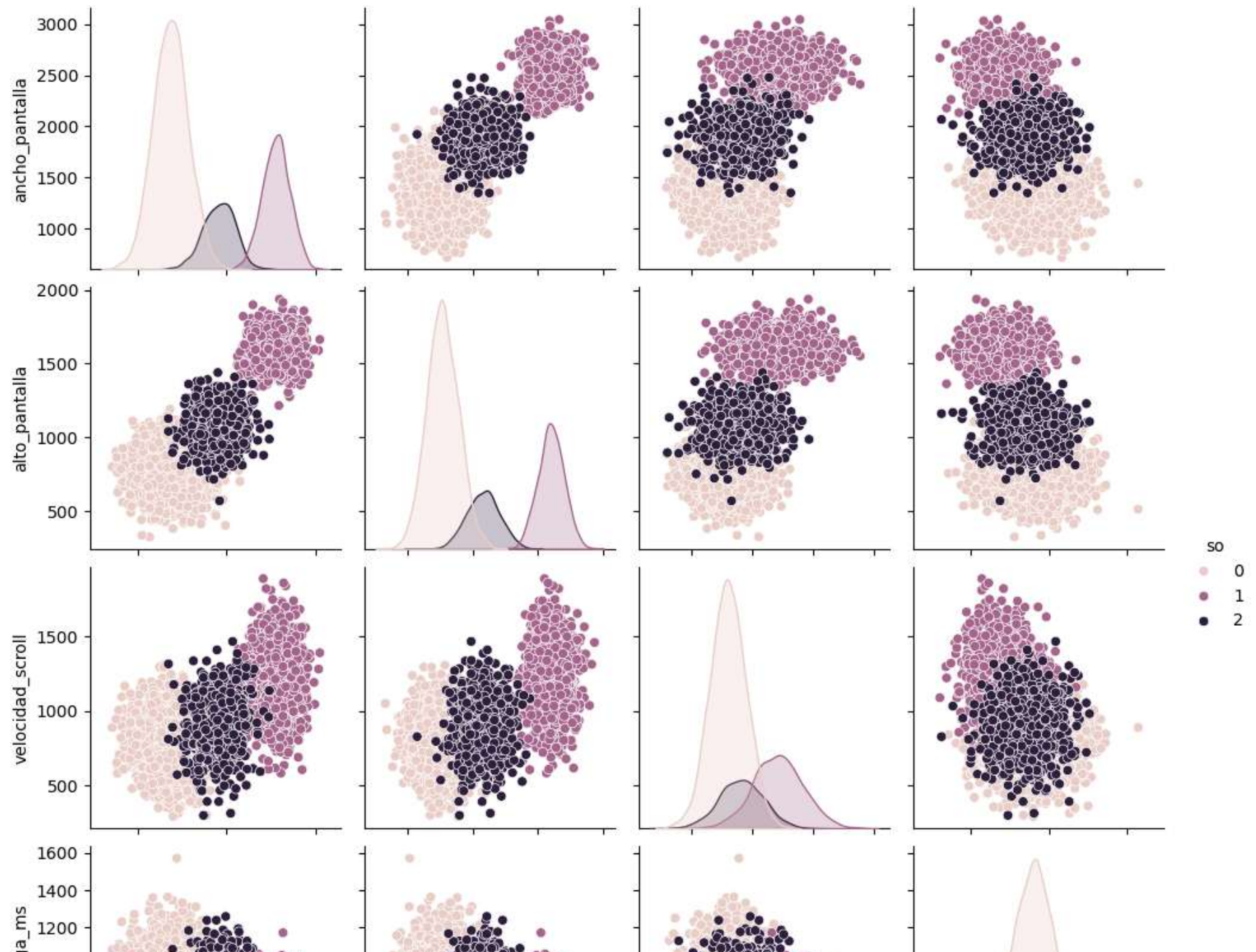
so

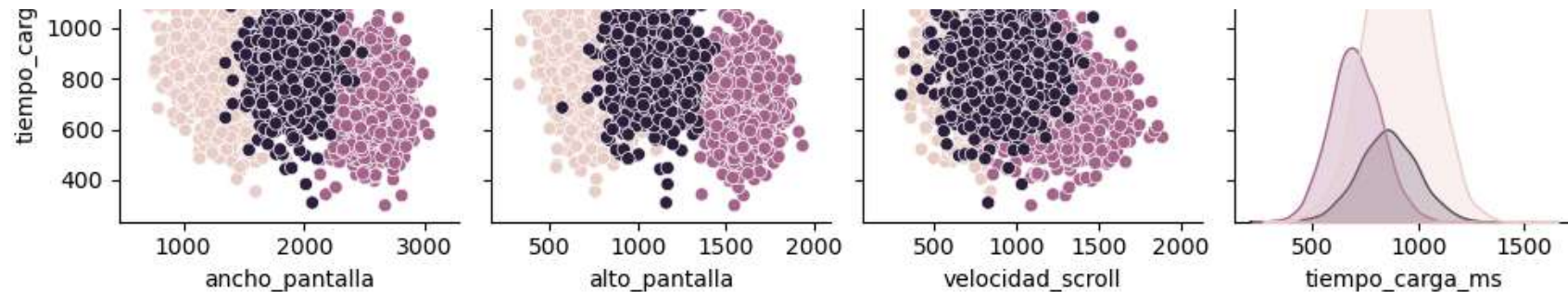
0 3000

1 1250

2 750

Name: count, dtype: int64





5.- División Train/Test

Dividimos los datos en entrenamiento (80%) y prueba (20%).

```
In [27]: X = data[['ancho_pantalla', 'alto_pantalla', 'velocidad_scroll', 'tiempo_carga_ms']]
y = data['so']
X_train, X_test, y_train, y_test = model_selection.train_test_split(
X, y, test_size=0.2, random_state=42
)
X_train.shape, X_test.shape
```

```
Out[27]: ((4000, 4), (1000, 4))
```

6.- Entrenamiento del modelo de regresión logística

```
In [28]: # Importamos una herramienta que ajusta todas las variables a la misma escala
# Esto ayuda a que el modelo aprenda mejor y no dé más importancia a variables con números grandes
from sklearn.preprocessing import StandardScaler

# Creamos el "escalador", que es el objeto encargado de hacer esa transformación
scaler = StandardScaler()

# Ajustamos el escalador con los datos de entrenamiento y los transformamos
# (aprende cómo son los datos y los pone todos en la misma escala)
X_train_scaled = scaler.fit_transform(X_train)

# Transformamos los datos de prueba usando la misma escala aprendida
```

```

# (muy importante para no mezclar información entre entrenamiento y prueba)
X_test_scaled = scaler.transform(X_test)

# -----
# ENTRENAMIENTO DEL MODELO MULTINOMIAL
# -----

# Creamos un modelo de regresión logística para clasificar en varias clases
# - solver='lbfgs' es un método que funciona bien con varios grupos
# lbfgs (siglas de Limited-memory Broyden-Fletcher-Goldfarb-Shanno) es un método matemático que encuentra los coeficientes que
# - max_iter=1000 asegura que el modelo tenga tiempo suficiente para aprender
modelo = linear_model.LogisticRegression(
    solver='lbfgs',
    max_iter=1000
# La regresión logística, se ajusta iterativamente. Cada iteración mejora la estimación de coeficientes.
# Si hay muchas, el modelo tiene más oportunidades de encontrar la solución óptima
)

# Entrenamos el modelo usando los datos ya escalados
modelo.fit(X_train_scaled, y_train)

# Mostramos los coeficientes que el modelo ha aprendido
print("Coeficientes del modelo:")
print(modelo.coef_)

# Mostramos el valor independiente (bias) del modelo
print("\nIntercept:")
print(modelo.intercept_)

```

Coeficientes del modelo:

```

[[-5.06226328 -5.27659908 -0.74242871  0.60764299]
 [ 4.30687823  5.13599462  0.83305825 -0.74368374]
 [ 0.75538505  0.14060446 -0.09062954  0.13604075]]

```

Intercept:

```
[ 1.68694288 -4.98072312  3.29378024]
```

Explicación del escalado de variables (tipificar la variable normal es transformar $N(\mu, \sigma)$ en una $N(0,1)$)

El *escalado de variables* consiste en transformar todas las características numéricas para que tengan la **misma escala**, evitando que unas dominen sobre otras debido a diferencias en sus magnitudes. En este caso se utiliza `StandardScaler`, que aplica una **estandarización**:

$$X_{\text{scaled}} = \frac{X - \mu}{\sigma}$$

donde:

- μ es la media de cada variable
- σ es su desviación estándar

Tras esta transformación, cada variable queda con:

- **Media = 0**
- **Desviación estándar = 1**

Esto es especialmente importante en modelos como la **regresión logística**, que utilizan métodos de optimización numérica. Si las variables están en escalas muy diferentes (por ejemplo, píxeles, milisegundos, velocidades), el modelo puede:

- Converger más lentamente
- Dar más peso a variables con valores grandes
- Obtener coeficientes distorsionados

Por eso se realiza:

```
scaler = StandardScaler()  
X_train_scaled = scaler.fit_transform(X_train)  
X_test_scaled = scaler.transform(X_test)
```

- `fit_transform()` calcula la media y desviación estándar **solo con los datos de entrenamiento** y los transforma.
- `transform()` aplica esa misma transformación al conjunto de prueba, evitando *data leakage*.

Una vez escalados los datos, el modelo se entrena de forma más estable y eficiente:

```
modelo = linear_model.LogisticRegression(  
    solver='lbfgs',  
    max_iter=1000
```

```
)
modelo.fit(X_train_scaled, y_train)
```

El resultado son coeficientes más interpretables y un entrenamiento más robusto.

7.- Evaluación completa del modelo

```
In [29]: # EVALUACIÓN COMPLETA DEL MODELO
# 1. Predicciones usando los datos escalados
y_pred = modelo.predict(X_test_scaled)
# 2. Accuracy
print("Accuracy del modelo:", accuracy_score(y_test, y_pred))
# 3. Matriz de confusión
print("\nMatriz de confusión:")
print(confusion_matrix(y_test, y_pred))
# 4. Reporte de clasificación
print("\nReporte de clasificación:")
print(classification_report(y_test, y_pred))
```

Accuracy del modelo: 0.98

Matriz de confusión:

```
[[592  0   6]
 [  0 252  0]
 [ 14   0 136]]
```

Reporte de clasificación:

	precision	recall	f1-score	support
0	0.98	0.99	0.98	598
1	1.00	1.00	1.00	252
2	0.96	0.91	0.93	150
accuracy			0.98	1000
macro avg	0.98	0.97	0.97	1000
weighted avg	0.98	0.98	0.98	1000

Evaluación del Modelo (Resumen Claro)

Accuracy

El modelo acierta el **98%** de las predicciones.
Rendimiento **excelente** para un problema con 3 clases.

Matriz de Confusión

Clase real ↓ / Predicha →	Windows	Mac	Linux
Windows (0)	592	0	6
Mac (1)	0	252	0
Linux (2)	14	0	136

Interpretación rápida:

- **Windows:** casi perfecto (solo 6 errores → confundidos con Linux).
- **Mac:** perfecto (0 errores).
- **Linux:** la clase más difícil (14 casos confundidos con Windows).

Reporte de Clasificación

Clase	Precision	Recall	F1-score	Interpretación
Windows (0)	0.98	0.99	0.98	Se identifica muy bien.
Mac (1)	1.00	1.00	1.00	Se clasifica perfectamente.

Clase	Precision	Recall	F1-score	Interpretación
Linux (2)	0.96	0.91	0.93	Algo más difícil; se confunde con Windows.

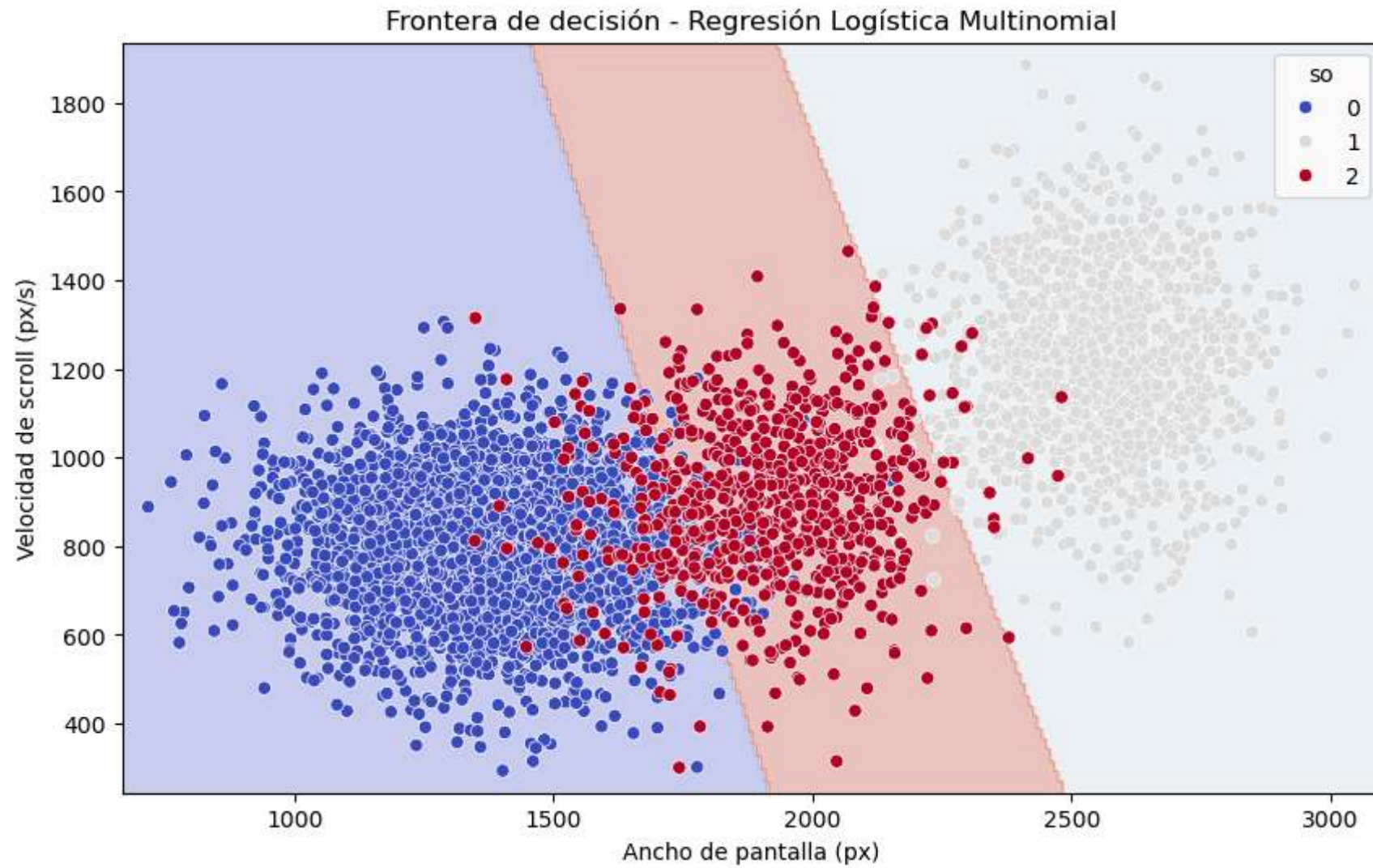
Conclusión General

- El modelo tiene un rendimiento **muy alto** (98% accuracy).
- **Mac** es la clase más fácil de identificar (100% aciertos).
- **Windows** también se clasifica con gran precisión.
- **Linux** presenta más solapamiento con Windows, por eso tiene un recall algo menor.
- En conjunto, el modelo es **fiable, estable y muy efectivo** para distinguir entre los tres sistemas operativos.

8.- Visualización de la frontera

```
In [30]: from sklearn.preprocessing import StandardScaler
# Seleccionamos solo dos variables para poder graficar
X2 = data[['ancho_pantalla', 'velocidad_scroll']]
y2 = data['so']
# Escalado de estas dos variables
scaler2 = StandardScaler()
X2_scaled = scaler2.fit_transform(X2)
# Entrenamos un modelo SOLO con estas dos variables
modelo2 = linear_model.LogisticRegression(
    solver='lbfgs',
    max_iter=1000
)
modelo2.fit(X2_scaled, y2)
# Crear malla de puntos para graficar la frontera
x_min, x_max = X2['ancho_pantalla'].min() - 50, X2['ancho_pantalla'].max() + 50
y_min, y_max = X2['velocidad_scroll'].min() - 50, X2['velocidad_scroll'].max() + 50
xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 300),
    np.linspace(y_min, y_max, 300)
)
```

```
# Convertimos La malla en DataFrame
grid = pd.DataFrame({
    'ancho_pantalla': xx.ravel(),
    'velocidad_scroll': yy.ravel()
})
# Escalamos la malla igual que los datos reales
grid_scaled = scaler2.transform(grid)
# Predicción sobre la malla
Z = modelo2.predict(grid_scaled)
Z = Z.reshape(xx.shape)
# Gráfico
plt.figure(figsize=(10, 6))
plt.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')
# Puntos reales
sb.scatterplot(
    data=data,
    x='ancho_pantalla',
    y='velocidad_scroll',
    hue='so',
    palette='coolwarm'
)
plt.title("Frontera de decisión - Regresión Logística Multinomial")
plt.xlabel("Ancho de pantalla (px)")
plt.ylabel("Velocidad de scroll (px/s)")
plt.show()
```



9.- Interpretación del modelo

```
In [31]: # INTERPRETACIÓN DE COEFICIENTES
coef_df = pd.DataFrame(modelo.coef_, columns=X.columns)
coef_df['Clase (SO)'] = ['Windows (0)', 'Macintosh (1)', 'Linux (2)']
coef_df
```

Out[31]:

	ancho_pantalla	alto_pantalla	velocidad_scroll	tiempo_carga_ms	Clase (SO)
0	-5.062263	-5.276599	-0.742429	0.607643	Windows (0)
1	4.306878	5.135995	0.833058	-0.743684	Macintosh (1)
2	0.755385	0.140604	-0.090630	0.136041	Linux (2)

Sistema Operativo	ancho_pantalla	Interpretación	alto_pantalla	Interpretación	velocidad_scroll	Interpretación	tiempo_carga_ms	Interpretación
Windows (0)	-5.06	Pantallas más grandes reducen mucho la probabilidad de Windows.	-5.28	Igual que el ancho: Windows se asocia a pantallas pequeñas.	-0.74	Scroll más rápido reduce la probabilidad de Windows.	+0.61	Tiempos de carga altos aumentan la probabilidad de Windows.
Macintosh (1)	+4.31	Pantallas grandes aumentan mucho la probabilidad de Mac.	+5.14	Mac se asocia fuertemente a pantallas altas.	+0.83	Scroll rápido aumenta la probabilidad de Mac.	-0.74	Tiempos de carga bajos aumentan la probabilidad de Mac.
Linux (2)	+0.76	Ligera preferencia por pantallas algo más grandes.	+0.14	Influencia muy baja.	-0.09	Prácticamente no influye.	+0.14	Influencia muy baja.

```
In [ ]:
```